

Introducción a la Programación I

Clase 4: Funciones y Scope

¿Qué es una función?

Una **función** es un bloque de código con un nombre, diseñado para realizar una tarea específica.

- Se define **una vez** y se puede usar **muchas veces**
- Hace que el código sea más legible y organizado
- Es la base de la programación modular

```
# Ejemplo simple
def saludar():
    print("¡Hola, mundo!")

saludar() # llamada a la función
```

¿Por qué usar funciones?

- ✓ **Reutilización:** escribís el código una vez, lo usás donde quieras
- ✓ **Legibilidad:** nombres descriptivos hacen el código más claro
- ✓ **Testing:** podés probar cada función por separado
- ✓ **Mantenimiento:** si hay un error, lo corregís en un solo lugar

Funciones built-in de Python

Python ya trae muchas funciones listas para usar:

```
print("Hola")           # imprime en pantalla
len("Python")           # longitud → 6
type(42)                # tipo de dato → <class 'int'>
int("10")               # convierte a entero → 10
float("3.14")           # convierte a float → 3.14
str(100)                # convierte a string → "100"
input("Tu nombre: ")    # lee entrada del usuario
range(5)                # genera secuencia 0..4
abs(-7)                 # valor absoluto → 7
round(3.7)              # redondea → 4
```

Estas son exactamente el tipo de cosas que vamos a poder crear nosotros.

Definir una función

```
def saludar():  
    """Imprime un saludo en pantalla."""  
    print("¡Hola!")
```

Parte	Descripción
def	keyword que indica "definir función"
saludar	nombre de la función
()	paréntesis (obligatorios)
:	dos puntos para abrir el bloque
Cuerpo indentado	las instrucciones de la función
"""..."""	docstring: descripción opcional pero recomendada

Llamar a una función

Definir una función **no la ejecuta**. Hay que **llamarla**:

```
def saludar():  
    """Imprime un saludo en pantalla."""  
    print("¡Hola!")  
  
# Definición arriba ↑ – todavía no pasó nada  
  
saludar()    # ← acá se ejecuta  
saludar()    # ← se puede llamar cuantas veces quieras  
saludar()
```

Salida:

```
¡Hola!  
¡Hola!  
¡Hola!
```

Pasar información a una función

Podemos hacer que una función reciba datos usando **parámetros**:

```
def saludar(nombre):  
    """Saluda a la persona indicada."""  
    print(f"¡Hola, {nombre}!")  
  
saludar("Ana")  
saludar("Carlos")  
saludar("María")
```

Salida:

```
¡Hola, Ana!  
¡Hola, Carlos!  
¡Hola, María!
```

`nombre` es el **parámetro**: una variable local que recibe el valor cuando se llama la función.

Argumentos vs Parámetros

Esta distinción es importante:

Concepto	Dónde aparece	Ejemplo
Parámetro	En la definición de la función	<code>def saludar(nombre)</code>
Argumento	En la llamada a la función	<code>saludar("Ana")</code>

```
def saludar(nombre):    # ← nombre es PARÁMETRO
    print(f"¡Hola, {nombre}!")

saludar("Ana")          # ← "Ana" es ARGUMENTO
```

El parámetro es el "casillero", el argumento es el "valor que ponemos en ese casillero".

Argumentos posicionales

El orden de los argumentos importa:

```
def describir_mascota(tipo, nombre):  
    """Describe una mascota."""  
    print(f"Tengo un {tipo} que se llama {nombre}.")  
  
describir_mascota("perro", "Rex")  
describir_mascota("gato", "Michi")
```

Salida:

```
Tengo un perro que se llama Rex.  
Tengo un gato que se llama Michi.
```

⚠ Si invertís el orden, el resultado cambia:

```
describir_mascota("Rex", "perro") # ← Tengo un Rex que se llama perro.
```

Argumentos por keyword

Podés especificar a qué parámetro va cada valor usando el nombre:

```
def describir_mascota(tipo, nombre):  
    print(f"Tengo un {tipo} que se llama {nombre}.")  
  
describir_mascota(tipo="perro", nombre="Rex")  
describir_mascota(nombre="Michi", tipo="gato") # ← orden no importa
```

Ambas líneas producen el mismo resultado. Los **argumentos por keyword** son útiles cuando:

- La función tiene muchos parámetros
- Querés que el código sea más explícito y legible

Múltiples llamadas

Una de las grandes ventajas de las funciones: reutilizar con distintos datos.

```
def saludar(nombre):  
    print(f"¡Hola, {nombre}!")  
    print(f"Bienvenido/a al curso, {nombre}.\n")  
  
saludar("Ana")  
saludar("Carlos")  
saludar("María")
```

Salida:

```
¡Hola, Ana!  
Bienvenido/a al curso, Ana.  
  
¡Hola, Carlos!  
Bienvenido/a al curso, Carlos.  
...
```

Valores de retorno: `return`

Una función puede **devolver un resultado** con `return`:

```
def get_nombre_completo(nombre, apellido):  
    """Devuelve el nombre completo formateado."""  
    nombre_completo = f"{nombre} {apellido}"  
    return nombre_completo.title()  
  
alumno = get_nombre_completo("juan", "pérez")  
print(alumno)    # Juan Pérez
```

- `return` termina la ejecución de la función
- El valor devuelto se puede guardar en una variable o usar directamente
- Sin `return`, la función devuelve `None`

Ejemplo completo con return

```
def get_nombre_completo(nombre, apellido):  
    """Devuelve el nombre completo formateado."""  
    return f"{nombre.title()} {apellido.title()}"  
  
# Guardamos el resultado  
nombre1 = get_nombre_completo("ana", "garcía")  
nombre2 = get_nombre_completo("carlos", "lópez")  
  
print(nombre1)    # Ana García  
print(nombre2)    # Carlos López  
  
# O usamos el resultado directamente  
print(get_nombre_completo("maría", "martínez")) # María Martínez
```

Scope en Python

Scope (ámbito) es el área del programa donde un nombre (variable, función) es accesible.

Python tiene dos scopes principales:

Scope	Dónde se define	Dónde es visible
Global	Nivel superior del programa	En todo el código
Local	Dentro de una función	Solo dentro de esa función

Scope global

Las variables definidas fuera de cualquier función son **globales**:

```
mensaje = "Hola desde el nivel global"    # variable global

def mostrar():
    print(mensaje)    # puede leer la variable global

mostrar()            # Hola desde el nivel global
print(mensaje)       # Hola desde el nivel global
```

Las variables globales son visibles **desde cualquier parte** del código.

Scope local

Las variables creadas **dentro** de una función son **locales**:

```
def calcular_cuadrado(base):  
    resultado = base ** 2    # variable local  
    return resultado  
  
print(calcular_cuadrado(5))    # 25  
  
# Esto falla:  
print(resultado)    # NameError: name 'resultado' is not defined  
print(base)         # NameError: name 'base' is not defined
```

- Las variables locales se **crean** cuando se llama la función
- Se **destruyen** cuando la función termina
- No existen fuera de la función

La keyword `global`

Para modificar una variable global desde dentro de una función, usamos `global`:

```
contador = 0    # variable global

def incrementar():
    global contador    # le decimos a Python que use la variable global
    contador += 1

incrementar()
incrementar()
incrementar()
print(contador)    # 3
```

Sin `global`, Python crearía una nueva variable local llamada `contador` y la global no cambiaría.

Cuidado con `global`

⚠ Si declaras una variable como `global` dentro de una función pero **no llamas la función**, la variable no existirá:

```
def crear_nombre():  
    global nombre  
    nombre = "Ana"  
  
# Si no llamamos crear_nombre():  
print(nombre)    # NameError: name 'nombre' is not defined  
  
# Hay que llamar la función primero:  
crear_nombre()  
print(nombre)    # Ana
```

Variables globales: cuándo usarlas

Las variables globales son útiles para:

- Constantes que no cambian (convención: MAYÚSCULAS)
- Configuración del programa

```
VELOCIDAD_LUZ = 299_792_458    # constante global
PI = 3.14159

def calcular_circunferencia(radio):
    return 2 * PI * radio
```

Evitá las variables globales mutables cuando:

- Muchas funciones las modifican (difícil de depurar)
- El código se vuelve difícil de entender y testear
- Preferí pasar valores como argumentos y usar `return`

Convención `_` para "privado"

En Python no existe la privacidad estricta como en otros lenguajes, pero hay una convención:

```
_variable_interna = 42      # convención: "no usar desde afuera"  
__nombre_muy_interno = 99  # doble guión: name mangling en clases  
  
def _funcion_auxiliar():  
    """Esta función es de uso interno."""  
    pass
```

- Un `_` al inicio es una señal para otros programadores: *"esto es de uso interno"*
- Python **no lo impide**, es solo una convención
- Es especialmente relevante cuando trabajamos con módulos y clases (temas futuros)

Resumen

Concepto	Clave
<code>def nombre():</code>	Define una función
Docstring <code>"""..."""</code>	Documenta qué hace la función
Parámetro	Variable en la definición
Argumento	Valor en la llamada
Argumento posicional	Orden importa
Argumento keyword	<code>nombre=valor</code> , orden no importa
<code>return</code>	Devuelve un valor y termina la función
Scope global	Visible en todo el programa
Scope local	Solo dentro de la función